

Forward Assignment Rule (continue)

- **[Forward Assignment Rule]** $p[v_0 / v] \wedge v = e[v_0 / v]$ is a postcondition for *partially valid triple* $\{p \wedge v = v_0\} v := e \{...\}$. Note that, even if we omit the $v = v_0$ in the precondition, it can still be understood.
- 1. Create a valid triple $\{x > 0\} x := x - 1 \{...\}$.
 - With the forward assignment rule, we can age the variable that is being assigned to, then we have:

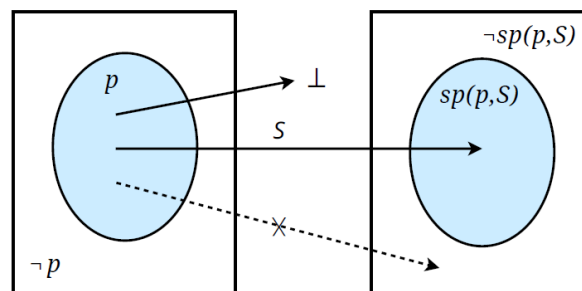
$$\{x > 0 \wedge x = x_0\} x := x - 1 \{x_0 > 0 \wedge x = x_0 - 1\}$$
 - We can use existential to represent the postcondition to remove the aging of x in the precondition. The following triple is still valid:

$$\{x > 0\} x := x - 1 \{\exists x_0. x_0 > 0 \wedge x = x_0 - 1\}$$
 - x_0 only appears in the postcondition; so, dropping of the existential quantifier and only keeping the body already implies the existence of such x_0 . Then we have:

$$\{x > 0\} x := x - 1 \{x_0 > 0 \wedge x = x_0 - 1\}$$
- From the above example, we can see that it is okay to omit the aging part in the precondition and use fresh logic variable x_0 for x in the postcondition directly to represent the old value of variable x . It is still understood.

The Strongest Postconditions

- Given a precondition p and program S , the **strongest postcondition** of p and S , written as $sp(p, S)$ is (the predicate that stands for) the set of states we could terminate in, if we run S starting in a state that satisfies p .
 - In symbols, using the language of states: $sp(p, S) = \{\tau \mid \tau \in M(S, \sigma) - \perp \text{ for some } \sigma \text{ where } \sigma \models p\}$.
 - From the definition of the strongest postcondition we can see that $\models \{p\} S \{sp(p, S)\}$; in other words, this postcondition only guarantees a valid triple under partial correctness. The reason is easy to see: the precondition p is given, and there could be states that satisfy p make program S diverge or create error.



- From the above figure, we can see:
 - If $\sigma \models p$, then $\forall \tau. \tau \in M(S, \sigma) \rightarrow \tau = \perp \vee \tau \models sp(p, S)$.
 - If $\sigma \models \neg p$, we don't know anything interesting about $M(S, \sigma)$.

2. Prove that $\models \{p\} S \{q\} \Leftrightarrow (sp(p, S) \Rightarrow q)$.

It looks trivial, but our definition of $sp(p, S)$ didn't say anything about this postcondition the strongest. In this example, let us prove that $sp(p, S)$ is the strongest postcondition.

- $\Leftarrow$: By the definition of $sp(p, S)$, we have $\models \{p\} S \{sp(p, S)\}$. And since $sp(p, S) \Rightarrow q$, then $\models \{p\} S \{q\}$ (weakening the postcondition).
- $\Rightarrow$: Let τ be a state such that $\tau \models sp(p, S)$. By the definition of $sp(p, S)$, there exists some $\sigma \models p$ such that $\tau \in M(S, \sigma) - \perp$. $\models \{p\} S \{q\}$ implies that $M(S, \sigma) - \perp \models q$, thus $\tau \models q$. To sum up, we get "if $\tau \models sp(p, S)$, then $\tau \models q$ ", which implies " $sp(p, S) \Rightarrow q$ ".

Calculate sp for Loop-free Programs

Like wlp , we can use some algorithm/rules to calculate $sp(p, S)$ textually.

- $sp(p, \text{skip}) \equiv p$.
- $sp(p, v := e) \equiv p[v_0 / v] \wedge v = e[v_0 / v]$, where v_0 is the aged v (in other words, the old value of v before executing $v := e$).
 - This is the forward assignment rule, so actually this rule can produce the strongest postcondition.
- $sp(p, S_1; S_2) \equiv sp(sp(p, S_1), S_2)$.

3. Calculate the following sp 's.

- a. $sp(x > y, x := x + k) \equiv x_0 > y \wedge x = x_0 + k$
- b. $sp(x_0 > y \wedge x = x_0 + k, y := y + k) \equiv x_0 > y_0 \wedge x = x_0 + k \wedge y = y_0 + k$
- c. $sp(x > y, x := x + k; y := y + k) \equiv x_0 > y_0 \wedge x = x_0 + k \wedge y = y_0 + k$ #Combine a. and b.
 - By losing x_0 and y_0 , we can slightly weaken the postcondition to $x > y$.
- d. $sp(x > f(x, y), x := x + 1; x := x + x)$
 - $sp(x > f(x, y), x := x + 1) \equiv x_0 > f(x_0, y) \wedge x = x_0 + 1$
 - $sp(x > f(x, y), x := x + 1; x := x + x)$

$$\begin{aligned} &\equiv sp(x_0 > f(x_0, y) \wedge x = x_0 + 1, x := x + x) \\ &\equiv (x_0 > f(x_0, y) \wedge x = x_0 + 1)[x_1 / x] \wedge x = (x + x)[x_1 / x] \\ &\equiv (x_0 > f(x_0, y) \wedge x_1 = x_0 + 1) \wedge x = x_1 + x_1 \end{aligned}$$

- Let us think about sp in a conditional statement with an example:

$sp(T, \text{ if } x \geq y + z \text{ then } x := x - 1 \text{ else } y := y + 2 \text{ fi}) \equiv ?$

Following intuition, it is quite straightforward to come up with the following solution:

- When the if condition is true, we should have $sp(T \wedge x \geq y + z, x := x - 1) \equiv x_0 \geq y + z \wedge x = x_0 - 1$.
- When the if condition is false, we should have $sp(T \wedge x < y + z, y := y + 2) \equiv x < y_0 + z \wedge y = y_0 + 2$.
- The sp for the whole statement should one of the above, thus:
$$"sp"(T, \text{ if } x \geq y + z \text{ then } x := x - 1 \text{ else } y := y + 2 \text{ fi})$$

$$\equiv (x_0 \geq y + z \wedge x = x_0 - 1) \vee (x < y_0 + z \wedge y = y_0 + 2)$$
- Is this postcondition the strongest? No, it can be stronger since we didn't include that y is not updated in the true branch and x is not updated in the false branch. We need to add this information as well, and we need to be careful which variables are aged.

- To calculate the sp for a conditional statement, we need to calculate some variable sets first:
 - $lhs(S)$ = the set of variables that appear as the lhs of assignments in statement S .
 - $rhs(S)$ = the set of variables that appear as the rhs of assignments in statement S .
 - $free(p)$ = the set of variables that are free in precondition p .
 - $aged(p, S) = lhs(S) \cap (rhs(S) \cup free(p))$ is the set of variables whose assignments cause aging.
- Let $IF \equiv \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}$, and let $aged(p, IF) = \{x, y, \dots\}$. Then $sp(p, IF) \equiv sp(p_0 \wedge B, S_1) \vee sp(p_0 \wedge \neg B, S_2)$, where $p_0 \equiv p \wedge x = x_0 \wedge y = y_0 \dots$
- Let $NF \equiv \text{if } B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \text{ fi}$, and let $aged(p, NF) = \{x, y, \dots\}$. Then $sp(p, NF) \equiv sp(p_0 \wedge B_1, S_1) \vee sp(p_0 \wedge B_2, S_2)$, where $p_0 \equiv p \wedge x = x_0 \wedge y = y_0 \dots$

4. Calculate $sp(T, \text{if } x \geq y + z \text{ then } x := x - 1 \text{ else } y := y + 2 \text{ fi})$
 - Let $p \equiv T, S \equiv \text{if } x \geq y + z \text{ then } x := x - 1 \text{ else } y := y + 2 \text{ fi}$
 - $lhs(S) = \{x, y\}$
 - $rhs(S) = \{x, y\}$
 - $free(p) = \emptyset$
 - $aged(p, S) = \{x, y\}$
 - $sp(T \wedge x = x_0 \wedge y = y_0 \wedge x \geq y + z, x := x - 1)$
 $\equiv T \wedge x_0 = x_0 \wedge y = y_0 \wedge x_0 \geq y + z \wedge x = x_0 - 1$
 - $sp(T \wedge x = x_0 \wedge y = y_0 \wedge x < y + z, y := y + 2)$
 $\equiv T \wedge x = x_0 \wedge y_0 = y_0 \wedge x < y_0 + z \wedge y = y_0 + 2$
 - $sp(p, S)$
 $\equiv (x_0 = x_0 \wedge y = y_0 \wedge x_0 \geq y + z \wedge x = x_0 - 1) \vee (x = x_0 \wedge y_0 = y_0 \wedge x < y_0 + z \wedge y = y_0 + 2)$
 $\Leftrightarrow (y = y_0 \wedge x_0 \geq y + z \wedge x = x_0 - 1) \vee (x = x_0 \wedge x < y_0 + z \wedge y = y_0 + 2)$
5. Calculate $sp(p, S)$ where $p \equiv (x = y)$ and $S \equiv \text{if } y \geq 1 \rightarrow x := 1 \square y \leq 1 \rightarrow z := 0 \text{ fi}$.
 - $lhs(S) \equiv \{x, z\}$
 - $rhs(S) \cup free(p) \equiv \{x, y\}$
 - $aged(p, S) \equiv \{x\}$
 - $sp(x = y \wedge x = x_0 \wedge y \geq 1, x := 1) \equiv x_0 = y \wedge x_0 = x_0 \wedge y \geq 1 \wedge x = 1$
 - $sp(x = y \wedge x = x_0 \wedge y \leq 1, z := 0) \equiv x = y \wedge x = x_0 \wedge y \leq 1 \wedge z = 0$
 - $sp(p, S) \equiv (x_0 = y \wedge x_0 = x_0 \wedge y \geq 1 \wedge x = 1) \vee (x = y \wedge x = x_0 \wedge y \leq 1 \wedge z = 0)$